Graphical Modeling Tools for Systems Biology

ROSWITHA GOSTNER and BIANCA BALDACCI, The Microsoft Research - University of Trento Centre for Computational and Systems Biology
MELISSA J. MORINE and CORRADO PRIAMI, Dept of Mathematics, University of Trento and The Microsoft Research - University of Trento Centre for Computational and Systems Biology

Modeling biological systems to understand their mechanistic behavior is an important activity in molecular systems biology. Mathematical modeling typically requires deep mathematical or computing knowledge, and this limits the spread of modeling tools among biologists. Graphical modeling languages have been introduced to minimize this limit. Here, we survey the main graphical formalisms (supported by software tools) available to model biological systems with a primary focus on their usability, within the framework of modeling reaction pathways with two-dimensional (2D) (possibly nested) compartments. Considering the main characteristics of the surveyed formalisms, we synthesise a new proposal (Style) and report the results of an online survey conducted among biologists to assess usability of available graphical formalisms. We consider this proposal a guideline developed from what we learned in the survey, which can inform development of graphical formalisms to model reaction pathways in 2D space.

Categories and Subject Descriptors: I6.5 [**Simulation and Modelling**]: Model Development—*Modelling methodologies*; H5.2 [**Information Interfaces and Presentation**]: User Interfaces—*GUI*; D2.2 [**Software Engineering**]: Design Tools and Techniques—*User interfaces*

General Terms: Design, Languages

Additional Key Words and Phrases: Graphical modeling of biological systems, systems biology

ACM Reference Format:
Roswitha Gostner, Bianca Baldacci, Melissa J. Morine, and Corrado Priami. 2014. Graphical modeling tools for systems biology. ACM Comput. Surv. 47, 2, Article 16 (July 2014), 21 pages.
DOI: http://dx.doi.org/10.1145/2633461

1. INTRODUCTION

Understanding the interactions between the various components of the cell—such as DNA, RNA, and proteins—and how they assemble to form dynamic molecular systems is the foundation of systems biology [Maton et al. 1997]. There are two main analytical approaches in systems biology: static analysis (i.e., statistics and machine learning) and simulation-based analysis [Kitano 2002]. An example of a simulation-based approach is building a mathematical model to quantitatively describe the interactions between different molecules inside a cell, then simulate a perturbation of the system to gain knowledge about its dynamics [Westerhoff and Palsson 2004].

Computational modeling of cellular systems encompasses a broad range of related approaches, all carrying the goal of predicting complex, (non)linear behavior

This work has been supported by a grant of the Provincia Autonoma di Trento.
Authors' addresses: Roswitha Gostner, Bianca Baldacci, Melissa J. Morine, and Corrado Priami, COSBI, piazza Manifattura 1, 38068 Rovereto (TN), Italy; emails: priami@cosbi.eu; morine@cosbi.eu; r.gostner@gmail.com; baldaccibianca@gmail.com.
Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.
© 2014 ACM 0360-0300/2014/07-ART16 $15.00
DOI: http://dx.doi.org/10.1145/2633461

in cellular systems. The scope and biological function of such models ranges broadly, from modeling the comprehensive cellular dynamics of an entire single-celled organism [Karr et al. 2012; Rangamani et al. 2013] (many other references available at www.nrcam.uchc.edu/vcell_models/published_models.html), to modeling the behavior of a single membrane receptor [Sadiq et al. 2013]. These processes of the cell can be modeled in both time and space by considering the activity of molecular components, as well as their movement through spatial gradients, cell boundaries, and compartments [Kholodenko 2006] (see also www.ebi.ac.uk/biomodels-main/publmodels for a large set of computational models).

Biological pathway modeling represents a fundamental aspect of cellular behavior and thus of computational cellular modeling. Signaling, regulatory, and metabolic pathways characterize the primary classes of pathways, differing in their basic biological function and therefore in their biochemical content. Whereas signaling pathways typically consist of signaling molecules, receptors, and secondary messengers, regulatory networks consist of transcription factors and target DNA, and metabolic pathways comprise metabolic enzymes and metabolites. And, in reality, metabolic, signaling, and regulatory pathways interact and intersect in the course of cellular growth and activity. Regardless of the type of pathway being considered, the framework of a pathway model consists of the pathway elements (metabolites, enzymes, etc.) linked together via known functional interactions, each of which must be described by kinetic parameters (either observed or inferred) in order to apply computational modeling methods. Pathway modeling is fundamental for better understanding biological mechanisms that give rise to higher level phenotypes, as demonstrated, for example, by Cheong et al. [2008].

The focus of this review is on graphical representations of reaction pathways within two-dimensional (2D; possibly nested) compartments. As such, these representations can be interpreted as graphical programming languages, and compiler theory can be applied to compile/interpret the graphical language into another one that is supported by runtime simulators.

There are different modeling styles, for example traditional programming languages like C-family or Python, mathematical formalisms like ordinary differential equations, Petri nets or chemical reactions, rule-based formalisms or process algebras, and diagrammatic notations [Priami 2009].

Figure 1 presents a set of freely available modeling software tools. We provide a classification of the tools according to the available documentation, the support for domain-specific languages to write models, and the support for graphical modeling. The minimal requirement set for consideration was peer-reviewed publication, the ability to perform dynamic simulations with the software, and an active web page providing the software for download. Free packages for Matlab or Mathematica are not listed because they require commercial software to run. This article addresses graphical formalisms and usability of modeling tools. Hence, we focus on the tools that received the highest marks in the third column of Figure 1.

The literature reports graphical representations containing inconsistencies and redundancies that obfuscate communication across different research groups. A salient example is in Le Novère et al. [2009, figure 1], where the authors show how the same graphical symbol can be associated with eight different meanings and how a semantic concept can be represented with nine different graphical symbols. The same paper reports the efforts of a consortium that recently addressed this limitation and proposes the Systems Biology Graphical Notation (SBGN). SBGN is a standardized set of symbols with an agreed semantic to facilitate communication of models, the same way that UML did for software engineering [Le Novère et al. 2009].

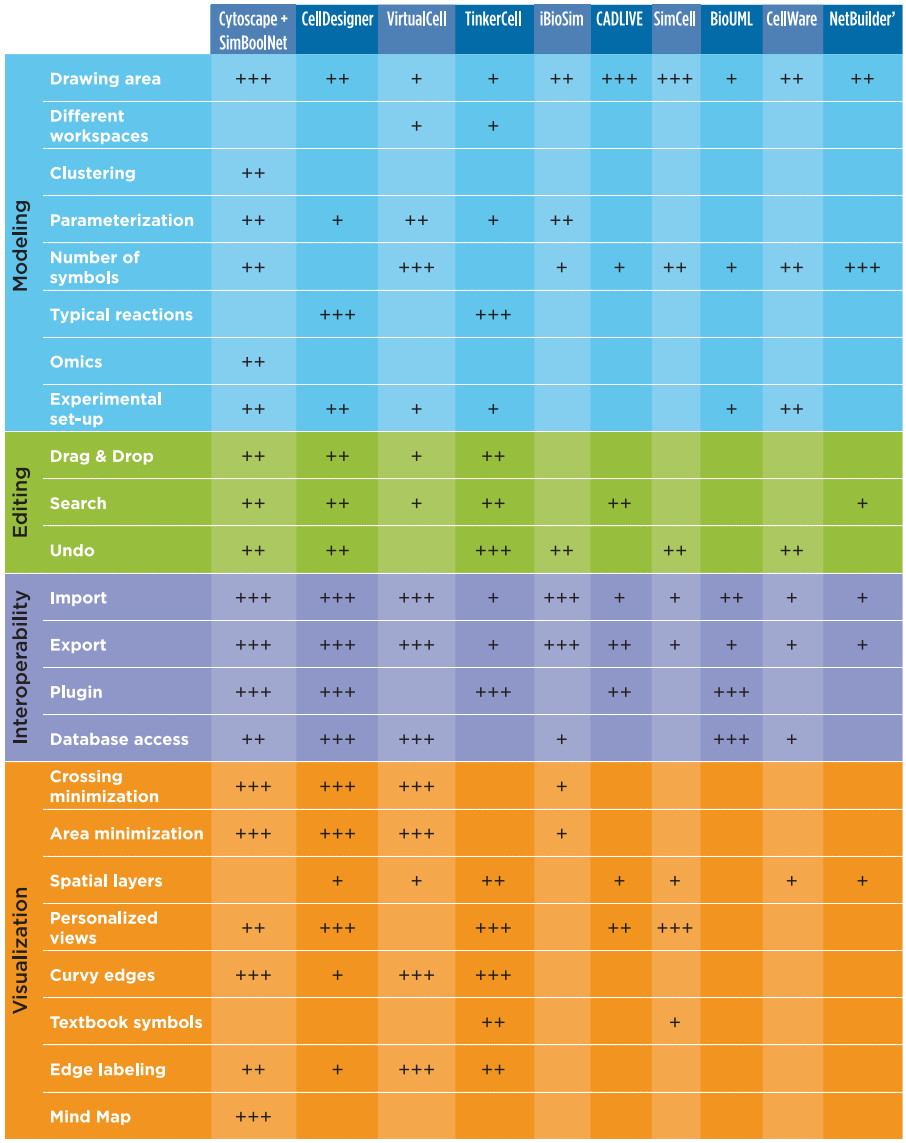
A diagrammatic notation focuses on the description of the system while a computer performs the translation into a suitable language for computer simulation. Starting

Tool	Documentation	DSL	Graphical
ALC [Koschorreck and Gilles 2008]	++	++	
ANC [Ollivier et al. 2010]	++	++	
BCT [Bray et al. 1993]	+	++	
BetaWB [Dematté et al. 2008]	++	+	++
BioNet [Bosl 2007]	++*		
BioNetGen [Colvin et al. 2009]+RuleBender [Smith et al. 2012]	++*	+++	+
Bio-PEPA [Ciocchetta et al. 2009]	++	+	
BioSPI [Priami et al. 2001]	++	+	
BioSpice [Garvey et al. 2003]	++	+++	
BioTapestry [Longabaugh 2012]	++	+++	++
BioUML [Kolpakov et al. 2006]	++		++
BoolNet [Müssel et al. 2010]	++*		
CADLIVE [Kurata et al. 2007]	++		+++
CellDesigner [Funahashi et al. 2008]	++		+++
CellLine [Ribeiro et al. 2007]	+	+	
CellWare [Dhar et al. 2004]	++		+++
Circadian Modelling [Akman et al. 2010]	++	+++	
CoPasi [Hoops et al. 2006]	++*	+++	
Cytoscape+SimBoolNet [Zheng et al. 2010]	++		+++
DBSolve Optimum [Gizzatkulov et al. 2010]	++	+	+
Dizzy [Ramsey et al. 2005]	++	++	
FERN [Erhard et al. 2008]	++*		
Gepasi [Mendes 1993]	++	+++	
iBioSim [Myers et al. 2009]	++		+++
JigCell [Vass et al. 2006]	++	+++	
JSIM [Miller et al. 2000]	++	+++	
JWSOnline[Olivier and Snoep 2004]	+	+++	
KinSolver [Aleman-Meza et al. 2009]	+	+++	
little b [Mallavarapu et al. 2007]	++	+	
MCell [Casanova et al. 2001]	++*	+++	+
Moose [Dudani et al. 2009]	++*	+++	+
NetBuilder' [Wegner et al. 2007]	++		+++
NFsim [Bachman and Sorger 2011]	++	+	
Odefy [Krumsiek et al. 2010]	++*		
PNK [Weber and Kindler 2003]	++*	+	++
PyDSTool [Clewley 2012]	++*		
PySB [Lopez et al. 2013]	++*	+	
SGN Sim [Ribeiro and Lloyd-Price 2007]	++	+	
SimCell [Ribeiro and Lloyd-Price 2007]	+		+++
Simmune [Meier-Schellersheim et al. 2006]	++	+++	+
Simpathica [Mishra et al. 2005]	+*		
SloppyCell [Myers et al. 2007]	++*		
Smoldyn [Andrews et al. 2010]	++*	++	
SPIM [Phillips 2009]	++	+	
SRSim [Grünert and Dittrich 2011]	+	+	
STOCKSIM [Le Novere and Shimizu 2001]	++	++	
STOCK [Kierzek 2002]	++	+	
SynBioss [Hill et al. 2008]	++	+++	
Systems Biology Workbench [Bergmann and Sauro 2006] +JDesigner+Jarnac	++*	+++	++
TinkerCell [Chandran et al. 2009]	++*		+++
VirtualCell [Moraru et al. 2008]	++	+++	+++
WebCell [Lee et al. 2006]	+	+++	+

Fig. 1. Classification of modeling tools. The ranking in the Documentation column is: (+) means that documentation is available and (++) also tutorials and examples are available; (*) means that the tool can be accessed as a standard library. The ranking in the DSL (Domain Specific Language) column is: (+) a DSL is available and (++) it is biology oriented, such as chemical reactions, (+++) means that the tool offers more than a text file as input, such as spreadsheet-like entry. The Graphical column evaluates display capabilities: (+) means that the tool can visualize models and (++) edit them, (+++) means that the tool can import standard formats such as SBML or SBGN. Tools with +++ in the Graphical column qualify for the detailed analysis.

from the analysis of pros and cons of the surveyed tools and other general-purpose graphical tools, we design a graphical language (Style) that can be automatically mapped to reaction-based or programming language-based representations of pathways with 2D compartments (our main sources of modeling are therefore protein-protein interaction networks, metabolic networks, and gene-regulatory networks). We consider our proposal as a guideline for the development of tools for graphical modeling of reaction pathways with 2D compartments that can be unambiguously mapped into formal representations suitable for simulation. We further validate the design choices of Style through an online survey among biologists, of which we report the results.

The article is organized as follows. The next section concentrates on graphical languages. We then propose a new language as the synthesis of the features of the existing biology-specific and general-purpose solutions in Section 3. This new language has been evaluated through an online survey, discussed in Section 4.



		Cytoscape + SimBoolNet	CellDesigner	VirtualCell	TinkerCell	iBioSim	CADLIVE	SimCell	BioUML	CellWare	NetBuilder'
Modeling	Drawing area	+++	++	+	+	++	+++	+++	+	++	++
	Different workspaces			+	+						
	Clustering	++									
	Parameterization	++	+	++	+	++					
	Number of symbols	++		+++		+	+	++	+	++	+++
	Typical reactions		+++		+++						
	Omics	++									
	Experimental set-up	++	++	+	+				+	++	
Editing	Drag & Drop	++	++	+	++						
	Search	++	++	+	++		++				+
	Undo	++	++		+++	++		++		++	
Interoperability	Import	+++	+++	+++	+	+++	+	+	++	+	+
	Export	+++	+++	+++	+	+++	++	+	+	+	+
	Plugin	+++	+++		+++		++		+++		
	Database access	++	+++	+++		+			+++	+	
Visualization	Crossing minimization	+++	+++	+++		+					
	Area minimization	+++	+++	+++		+					
	Spatial layers		+	+	++		+	+		+	+
	Personalized views	++	+++		+++		++	+++			
	Curvy edges	+++	+	+++	+++						
	Textbook symbols				++			+			
	Edge labeling	++	+	+++	++						
	Mind Map	+++									

Fig. 2. Each surveyed tool is listed in a column; features are listed in the rows and divided into four groups: modeling, editing, interoperability, and visualization. Tools are ranked from left to right according to the number of pluses in their respective column. The marks of the features distinguish four levels: no mark (missing feature), + (poor), ++ (average), and +++ (good).

2. GRAPHICAL MODELING TOOLS

Graphical modeling tools face a set of challenges with respect to the user interface in order to appear straightforward and easy to use; Figure 2 provides a summary of the tools we surveyed, focusing on 23 unique features (rows of the table). We group the characteristics into four groups: modeling support, editing, interoperability, and visualization of models.

2.1. Modeling

This subsection discusses user support to create biological models, associate them with quantitative parameters, and plan simulation experiments.

Modeling with a graphical language is an activity comparable to producing flow charts, video animations, or illustrations in print media. While a user is in the creative production phase, the layout of many commercial graphical software is tuned to provide a large drawing canvas with panes of drawing tools around the screen edges. In contrast, many of the modeling tools we surveyed provide by default a canvas that takes up less than 80% of the screen. The best support in terms of drawing space was provided by CADLIVE, Cytoscape, and SimCell, followed by CellDesigner, CellWare, iBioSim, and NetBuilder. Tools, such as BioUML, SimCell, and TinkerCell, reduced considerably the drawing area, using more space to display information about model simulation or model analysis.

Graphical modeling tools are multifunctional. On one hand, they need to provide an interface to draw the actual model, whereas on the other hand, they have to support the user in setting simulation parameters, configuring simulation experiments, and analyzing simulation data. Multifunctional tools have been studied and developed for many years in user interface design (e.g., video or animation production software face similar challenges because they have to support users at different work phases throughout a production). These software tools provide the user with different workspaces targeted at specific activities: for example, an animation tool provides both a workspace for drawing and for writing action script (e.g., the Xcode programming environment supports graphical interface design and action coding associated to the view objects). Different workspace layouts are optimal solutions for multilevel activities assisted by detachable toolbox windows. Only TinkerCell and VirtualCell include basic workspace support, whereas the rest of the tools have no support for workspace layouts.

Mathematical models describing pathways tend to consist of many reactions. It is important that users can interactively aggregate a subset of nodes into a cluster, a common feature in graph tools. Clusters permit users to group complete subsets of nodes or reactions into a discrete subunit and create a macro-view of the complete model. Currently, only Cytoscape provides this feature.

Edges between two nodes of a graphical mathematical model represent a reaction and therefore can potentially display reaction details such as kinetic laws or stoichiometry. To start a simulation, each edge needs to be well-defined; that is, each edge needs a kinetic law specification (parametrization of the model). We considered visual parametrization from three perspectives: first, a user can directly access the reaction by clicking on the edge; second, more kinetic information is directly available in the graphical model via tool-tips or roll-over widgets; and, finally, the tool provides visual feedback on whether an edge is well-defined. Cytoscape, VirtualCell, and iBiosim provide a direct access to the reaction via a double-click on the edge. In addition, these tools display the reaction name directly above an edge.

Most graphical software apply a set of symbols to represent different types/classes of nodes in the model (e.g., enzymes, cytokines, metabolites, etc.). This is useful up to a point: the more symbol choices a user has, the harder the model becomes to decode. Therefore, a minimal alphabet of symbols generating a language still expressive enough to model most of the relevant biological systems is a desirable characteristic of a graphical formalism, and tools with a small alphabet are therefore rated more highly. We grouped the surveyed software into three classes: less than 10 symbols, between 10 and 25, more than 25 symbols. Anything beyond 30 was considered unusable.

Some reactions are frequent within the modeling process and could be seen as motifs or repeating macros. It would be useful if a user could add these combined reactions or

patterns in a fast and straightforward way to the model. Currently, only CellDesigner and provide support for this feature.

The newly coined word "-omics" refers to data types in molecular biology that are measured in a high-throughput manner; for instance genomics, transcriptomics, proteomics, or metabolomics. These platforms typically produce data on the order of thousands to millions of variables per sample and have become fundamental to studying biological processes from a "systems" perspective. To favor modeling in systems biology, it would therefore be advantageous to integrate omics datasets into modeling tools. Having experimental data directly related to a node, a single reaction, or a set of reactions allows users to compare quickly results with observation from the wet lab, such as using network analysis to identify active subnetworks in a data-driven way [Morine et al. 2011]. Currently, only Cytoscape provides plugins for handling omic data within the modeling process.

Simulation experiments need to be customized: a user should be able to select part or the complete model for simulation; then, it should be possible to configure experiment duration and highlight entities to be visualized after the simulation is completed. Currently, CellDesigner, CellWare, and Cytoscape provide the most flexible experiment design interface, followed by BioUML, TinkerCell, and VirtualCell.

2.2. Editing

This subsection summarizes the basic features supporting users in editing and manipulating models. Intelligent drag-and-drop support inside the model is a critical feature for usability. For example, a user should be permitted to select a group of elements inside the model and drag and drop those to another place. All tools supported simple drag and drop, hence the focus here was whether drag and drop could be extended to a selection or even to clusters or compartments within the model. CellDesigner, Cytoscape, and TinkerCell were particularly supportive of drag and drop.

Search functions permit a user to find elements and reactions in a large model. Moreover, advanced search functions permit users to search in subsections, such as labels or reaction. In addition, search allows the user to explore available metadata in the model and manipulate the model efficiently on a higher level by applying regular expressions. About half the surveyed tools supported a search function.

The Undo feature permits the safe execution of an operation because it permits the user to return to the prior state without losing any information. This feature is very common in a wide variety of software. We rated basic support as permitting a return to a single operation, whereas several returns were rated higher, and tools showing the history of commands were rated best. TinkerDell scored best followed by CellDesigner, Cytoscape, CellWare, iBioSim, and SimCell.

2.3. Interoperability

This subsection discusses features permitting tools to be combined with other modeling and analysis environments. Collaboration between different research groups and also between different steps of the modeling and simulation process means collaboration with different file formats. It is important that tools not only support proprietary format but also standards. The Systems Biology Mark-up Language (SBML) is an XML-based file format developed by a consortium including experts from computer science, mathematics, and biology [Hucka et al. 2003]. The Systems Biology Graphical Notation (SBGN) is closely related to SBML, but while SBML focuses to represent a model specification, SBGN is concerned with a standardized graphical representation for signaling pathways, metabolic networks, and gene regulatory networks [Le Novère et al. 2009]. To get a high import rating, tools have to support at least those two

standards and preferably others in the field, such as CellML. CellDesigner, Cytoscape, iBioSim, and VirtualCell are the most flexible tools with respect to file import.

Similarly to the import function, the export or saving features were investigated. Tools received a high rating if they supported open standards such as SBML and SBGN. Moreover, one aspect was whether tools supported export of graphical information into a graphical file format, such as JPEG or PNG, or, even better, a vector format such as SVG; such a feature is useful when users would like to integrate the model in a publication or presentation. CellDesigner, Cytoscape, iBioSim, and VirtualCell provide the best support.

Graphical modeling tools need to anticipate changes, and therefore their software architecture should permit extensions for customization. Current approaches employ plugin-based architecture, and therefore the survey investigated whether the graphical modeling tools provide a plugin structure supporting users to add extensions to the software. Supportive documentation, such as tutorials and examples for plugin, are also included because they are important to engage and encourage a community to write and extend existing software. Currently, there are four tools particularly supportive for plugin structures: BioUML, CellDesigner, Cytoscape, and TinkerCell.

Modeling is never a single activity, and tools supporting various file formats should permit access to model databases to allow users to download and even to reuse some components of other models. We rated tools for integrating access to model databases such as Kegg, BioCarta, or the BEL framework. The best tools for this feature are CellDesigner, BioUML, and VirtualCell.

2.4. Visualization

This subsection presents features to visualize models in an information-rich and friendly way. In its basic form, graphical languages consist of edges and nodes, where nodes represent entities within the cell and edges stand for an interaction between two or more entities. A mathematical model consists of several interactions, and it is therefore of great importance that edges are drawn in a well-arranged way, ideally avoiding crossing to follow the lines and identify reactions between entities at a glance. A graph drawn without edge crossing is called planar. Graph theory provides algorithms to automatically rearrange nodes so that edge crossing is minimized [Fleischer and Hirsch 2001; Purchase et al. 2001]. Moreover, there exist also algorithms that minimize the required drawing space for a graph layout [Huang et al. 2007; Frishman and Tal 2004]. Overall, an image looks clear and well-organized when nodes and edges fill the space homogeneously. The survey rates two aspects: minimizing edge crossing and minimizing required drawing area. There are several layout algorithms in CellDesigner, Cytoscape, and VirtualCell to arrange edges and nodes more efficiently.

A cell is formed of several well-known compartments, such as the cytoplasm, the nucleus, the mitochondria, and organelles. Graphical modeling tools should consider support for these structures. As models become larger, biological compartments provide a natural way to cluster and subdivide nodes. TinkerCell ranked best for this feature, permitting the definition of compartments, where elements could be added by drag and drop. Moreover, the tool permitted dragging a complete compartment around, with all its elements included.

Working on a large model, scientists like to group information by adding a customized color and/or shape scheme. For example, a set of proteins might be responsible for a common function, or omics data may indicate that a set of proteins has elevated abundance. Differentiating subgroups of proteins is another way to organize larger models and provide a high-level view. The survey considers whether users are able to customize single objects (such as a single protein, instance-based) or whether they can customize them on a class base (all proteins inside a model). Overall, the best

support is provided by CellDesigner, SimCell, and TinkerCell, which alow users to change background color and borders for each single element.

Graphical models describing biological processes will almost always have a set of edges crossing. The eye can easily follow the course of curvy edges, which is not the case for straight lines due to 90° cross angles. Curvy edges are available in tools such as PowerPoint or any flowchart drawing software. The survey investigates whether graphical modeling tools permit such organic flowing lines. CellDesigner, Cytoscape, TinkerCell, and VirtualCell support curvy edges. The low rating for CellDesigner is due to the fact that the line is not a single curved one but rather a set of single small straight lines, making the drawing easier to read but making the edge direction and angle particularly difficult to modify.

Illustrations of biological concepts are widely used in textbooks and journals. These cartoons are rich in information and have shaped the expectation for graphical modeling languages. Indeed, Saraiya et al. [2005] requirement analysis provides evidence that BioCarta is perceived as having the best pathway diagrams because its illustrations are very close to textbook style. The tool provides a variety of rounded forms and templates to indicate the different areas of a cell, such as cytoplasm, nucleus, or organelles. This survey rates tools depending on whether they use neutral illustration (like SBGN) or integrate biological context (e.g., plasma membrane, cytosol, mitochondria) and give different elements a different shape and place them in a logical reading format, like a biological textbook-style illustration (see, e.g., Alberts [2008]).

Graphical modeling tools that provide users with a variety of round-shaped elements, curvy edges, and a cell template are rated high for being textbook-style adherent. Currently, only TinkerCell and SimCell support symbols that lean toward textbook-style shapes.

The labeling of edges and nodes has an important impact on the aesthetic appearance of a graphical model. Graph theory provides algorithms to automatically find the best labeling mechanism in geographical informatics [Neyer 2001], which, however, is not the focus of this survey. Here, we are primarily concerned with how tools support the aesthetic aspect of labeling edges and nodes for increasing the readability and maintainability of models. For nodes, the challenging aspects are long protein names and how they are graphically resolved. Edges, on the other hand, are an ideal space to provide more details about kinetic reactions.

We considered how flexible the edge labeling is, such as providing details about the reaction directly on the edge. In addition, the survey considers visual feedback on whether an edge presents a well-defined reaction. Moreover, we consider whether edge labels adapt automatically to drag-and-drop gestures. These factors have not only an impact on the appearance of the graphical model, but also play a fundamental role in whether graphical models provide dense, rich information at a glance. Currently, best support is found in VirtualCell, followed by Cytoscape and TinkerCell.

Large models require scrolling because they no longer fit onto a canvas on the screen. Therefore, graphical modeling tools have to provide zoom and panning support to permit users to navigate the screen. Moreover, once the content is larger than the user's view field, tools must provide a means for acquiring and maintaining an overall map of the model structure [Eades et al. 1991].

Many graphical illustration tools approach this challenge by providing a thumbnail view of the complete structure outlining the view field the user sees on the screen: a so-called context- and -detail view. There is a large body of evidence that such context-and-detail views support the user in gaining an overall mind map of the graph's structure [Cockburn et al. 2008]. The survey evaluated whether graphical modeling tools provide support in navigating large models, looking for context and detailed views. Cytoscape

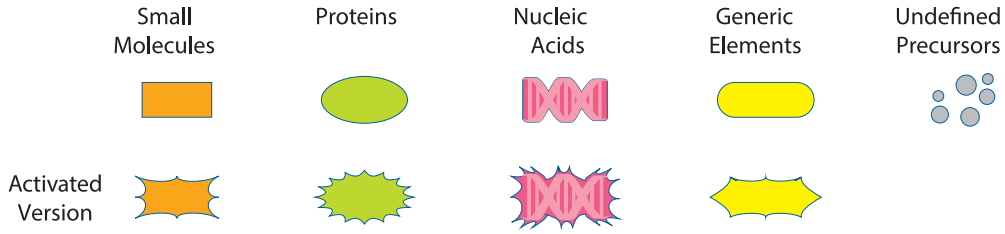


Fig. 3. The five species elements in Style; top row shows inactive elements, bottom row presents the same elements in their activated form.

permits a thumbnail view of the network and allows the user to navigate through the complete model by moving the viewpointCytoscape.

3. STYLE

Considerable effort has been made to produce a graphical language with a well-defined semantic, allowing researchers to write mathematical models of biological systems without the need to acquire programming skills. There has been little research in the actual choice of symbols, neither for the standard SBGN nor for the proprietary languages. We argue that current the approaches of graphical languages could be improved in two ways: first, by making the actual symbols less cluttered, and, second, by integrating symbols that are already familiar in the biological field. We present here a proposal of how an improved graphical language could look: the focus is on those symbols that should closely resemble to those already in use in biology and on lines and shapes that should be kept minimal in number to reduce unnecessary cluttering.

The graphical language Style has been designed by surveying biology textbooks and journal articles, combined with interviewing scientists in the field. A minimal set of symbols was obtained via an iterative process. Style consists of six space types, five element types, and three arrow types. We challenged the expressiveness of the language by creating a set of graphical models currently used by researchers at our center, ranging over signaling networks, metabolic networks, and gene regulatory networks.

One of the arguments that emerged for most existing graphical languages was their low information density [Saraiya et al. 2005]. As stated in the Introduction, Le Novère et al. [2009, figure 1] shows how the same graphical symbol can be used with different meanings and how a meaning can be associated with many different symbols.

In our design proposal for Style, we aimed for a small number of biologically meaningful symbols that are able to express a wide variety of biological models and for a graphical solution that is compact and similar to textbook illustrations.

3.1. Elements

Figure 3 presents the five elements of Style, namely small molecules, proteins, nucleic acids, generic elements, and undefined precursors. Each element has a distinctive shape and color, allowing the eye to immediately spot differences.

Informed by textbooks and interviews, we reserve the round shape for proteins (see green ellipse in Figure 3) and select a contrasting shape for small molecules (orange rectangle). Nucleic acids represent molecules carrying genetic information (genes or transcripts). These elements are depicted via a simplified double helix structure, outlining their close relationship with DNA. The undefined precursors are small elements from which cells synthesize or are peptides into which proteins degrade. In addition, Style offers a generic element, allowing the language to integrate custom components.

Each element comes in two versions: normal (top row in Figure 3) and spiky (bottom row). This graphical difference highlights a state change for a single element: for

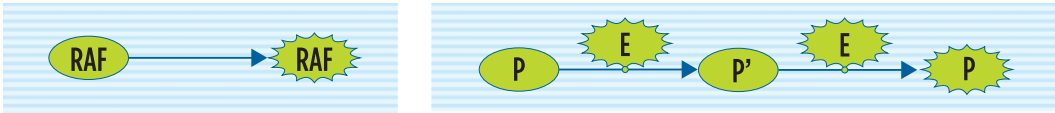


Fig. 4. Left: a reaction that makes an inactive element active. Right: example of enzymatic reactions where one substrate catalyzes two reactions.

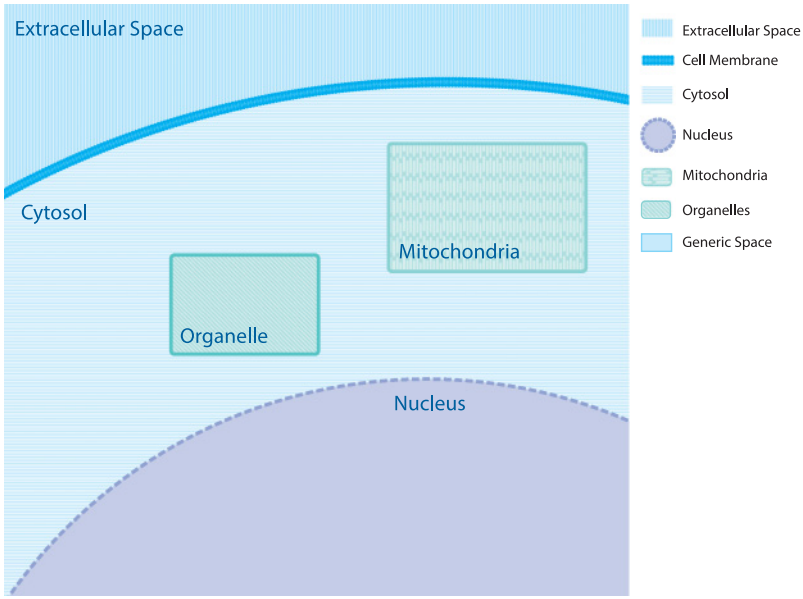


Fig. 5. Textures and patterns for illustrating different compartments within a cell. Each compartment is characterized by a volume and will be populated by the elements representing proteins, small molecules, DNA-related matter, generic elements, and raw material. Each element will be characterised by its concentration or number of molecules.

example, Figure 4 (left) shows an activation of the protein RAF and (right) a chain of phosphorylation, in which the element phosphorylates twice until it reaches the final configuration P″. Naming the final protein P″ in a round shape would work, but by changing only the shape and maintaining the same name, a user can express graphically that one is looking at the same protein but in a different state. Furthermore, spiky shapes have the potential to guide the eye immediately to interesting points within a pathway.

Style permits duplicates. A single element can influence more than one element: for example, in Figure 4 (right), the activated enzyme E influences two reactions. A solution alternative to the one depicted (admitting duplicates) is to represent each element only once in the pathway but use lines (similar to wiring lines in engineering) to indicate their related reactions (as implemented in CellDesigner). We exposed users to the wiring alternative, and they reported difficulties in understanding the meaning: their first guess was that the lines represent reactions rather than the influence of the enzyme on the reaction.

Informed by the requirement analysis published in Saraiya et al. [2005], Style proposes a simple cell template, with predefined elements such as extracellular space, cell membrane, cytosol, nucleus, mitochondria, organelles, and a generic element (Figure 5). All space elements use pastel colors for their pattern because their main purpose is to



Fig. 6. Style reactions. From left to right: modify, inhibit, and translocate.



Fig. 7. Approaches on how to handle graphically multiple catalyzers for a single reaction. In addition to the classical logical operators (the three cases on the left), the user can also define his own composing function (last on the right).

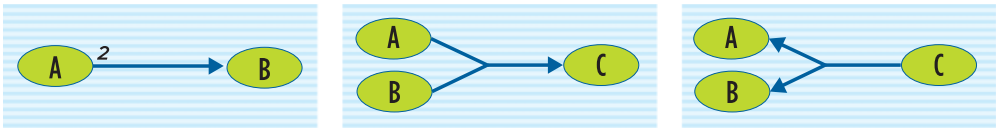


Fig. 8. Association and dissociation between two or more species. In the leftmost case, the number on the arrow close to protein A is the stoichiometry of the reaction (2 As are needed to form 1 B).

simply form background information. Chemical shapes or typical textbook illustration-inspired patterns and forms (e.g., the cell membrane) use a stylized phospholipid bi-layer.

3.2. Reactions

Style provides three distinct reaction types (Figure 6), namely modification (left), inhibition (middle), and translocation (right) between different cell compartments. Style uses the context of elements to break down their meaning into finer grained reactions, rather than introducing new types of arrows.

3.2.1. Modification. The simplest reaction, such as a substrate RAF becoming activated ($RAF \rightarrow RAF\ Activated$), can be expressed by connecting two shapes with a modify arrow, as in Figure 4 (left). If the reaction is performed with the aid of an enzyme, as for example in $P + E \rightarrow P' + E \rightarrow P\ Activated + E$, then the supporting enzyme is placed directly on the reaction arrow, as in Figure 4 (right). Style also permits a group of enzymes to influence a reaction by stacking elements one behind the other and relating them with a logical operator (Figure 7).

Modification reactions often require stoichiometry information, such as $A + A \rightarrow B$. For such cases, Style proposes an effective use of arrow labels indicating the required number of elements (Figure 8, left). Novel complexes are formed by association of two or more elements, for example $A + B \rightarrow C$. There, the graphical representation consists of two modify arrows merging into one, expressing an association (Figure 8, middle). When inverting the direction, the modify arrow expresses a dissociation (Figure 8, right). It is common practice to have unnamed complexes, meaning that an association does not have an explicit name. In such cases, Style duplicates the starting elements and surrounds them with a blue rectangle (Figure 9, left). Finally, reactions with co-factors, such as $A + B(cofactor) \rightarrow A'(product) + B'(cofactor\ modified)$, facilitated by the enzyme E, are still expressed via a modify arrow (Figure 9, right). As these examples illustrate, Style proposes context information to define the semantics of the language, thereby keeping the number of different symbols at a minimum.

3.2.2. Inhibition. In its simplest form, the inhibitor is attached to a reaction via a T-bar arrow (Figure 10, left). Should the reaction incorporate an enzyme, the same principle is applied (Figure 10, middle). Style allows the inhibition arrow to point directly to

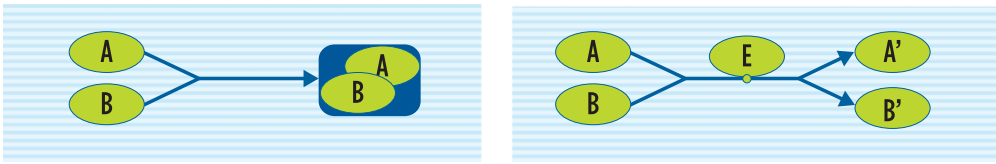


Fig. 9. Left illustrates how complexes are visualized. Right shows a reaction with co-factors.

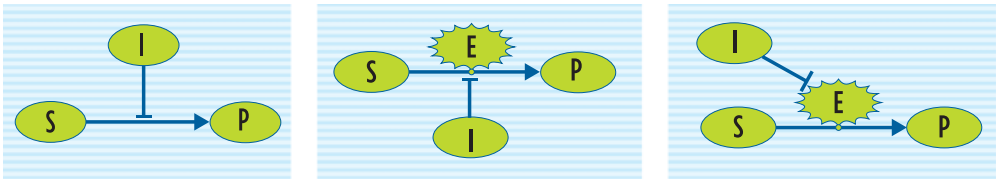


Fig. 10. Inhibition reactions: here, the element I inhibits the reaction between S and P.

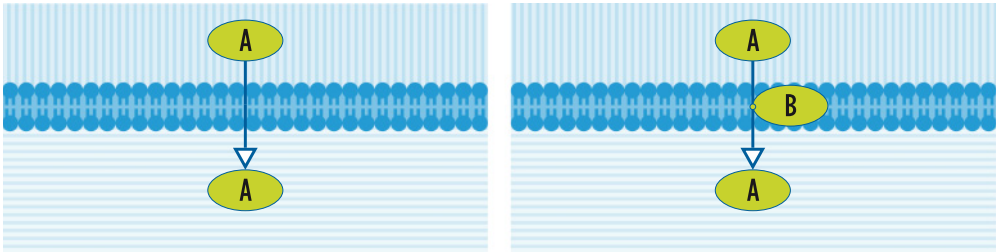


Fig. 11. Translocation of A through a membrane to a different compartment. Left: passive translocation; right: active translocation aided by the element B. Translocations can move an object from one compartment to another and in both directions. Note that the presence of the protein B within the membrane makes membranes first-class compartments as well.

the enzyme of a reaction (Figure 10, right) because this is often advantageous from a graphical point of view (although the middle and right of Figure 10 have semantically the same meaning). There are a range of different inhibitions, such as Michaelis-Menten (in its several forms, such as cooperative, competitive, and noncompetitive inhibition), Mass Action Law, and Custom-Defined Functions. Style aims to depict a general inhibition, and the exact details are omitted for clarity. The inhibition details are accessible via an interactive user interface applying concepts such as roll-over slides or tooltip information.

3.2.3. Translocation. Style proposes two types of translocation: when an element moves unaided from one space to another (passive transport, often gradient driven; see Figure 11, left) and when a protein/complex in a membrane transfers an element from one space to another (active transport; see Figure 11, right). It is important to note that the name of the translocating protein or complex remains the same.

4. ONLINE SURVEY

To assess the design of Style, we conducted an online survey. Before conducting the main survey, we performed a pilot study with a number of biologists not affiliated with our center to gain insight into the symbol choice in Style. Participants were given no explanation, and they could provide any answer in a short interview. These insights informed our final design but already indicated that Style's symbols and arrows were highly understandable. However, when designing the online survey, we decided to

restrict answers to a specific vocabulary (rather than allow free-form answers). Biology is a wide field, and different subgroups use different terminology. With a predefined set of answers, we forced participants to read and interpret a vocabulary following a certain direction, which should make the results more comparable.

The online survey consisted of three parts. First, participants had to rank three different graphical representations of the same pathway; second, they had to make an educated guess on elements of Style; and, finally, they had to answer questions regarding their computer literacy and statistical programming skills. The online survey was available for 2 months, from June until August 2012. There were 76 questionnaires completed with participants from the United Kingdom (59 participants), Italy (3), Germany (3), the United States (2), France, Hungary, India, Portugal, and Denmark. Half of the sampled participants were male, and half were female; the average age was 35.98 (SD = 10.48). Almost all (71 participants) had either an educational background or expertise in biology, systems biology, bioinformatics, or ecology. More than 22% held a master's degree, and more than two-thirds held a Ph.D. (17 participants).

While we designed the survey, the BioCarta server was down for several weeks, which forced us to investigate similar alternatives for implementing the comparison of three pathway illustrations. We identified Ingenuity Systems and Protein Lounge. Both incorporate textbook-style symbols and signs in their illustrations. We opted for Protein Lounge because it provides a 30-day trial; we were not able to access a trial version for Ingenuity Systems.

4.1. Results

This subsection reports the results of the online survey.

4.1.1. Ranking of Graphical Representation of Pathways. We chose a within-subject design with one independent variable: ease of reading, for which we tested each on CellDesigner, Protein Lounge, and Style. Users ranked three different pathways: gemcitabine metabolism, the folate cycle in mitochondria, and a subset of the folate cycle in cytosol (Figure 12). The reading order (from left to right) was randomized using Latin squares. Further, the order of questions was randomised too with Latin squares to minimize bias for both presentation sequence and reading order.

Participants had to rank the three cartoons with regard to the perceived "ease of read," with the easiest ranked first (see Figure 13). Applying a χ^2-test, there was a strong association for all three questions between the cartoon type and whether a participant ranked it as the easiest to read. Considering the three ranking questions corresponding to the three rows in Figure 12, the scores for the first question are $\chi^2(4) = 37.03$, $p < 0.001$; for the second question, $\chi^2(4) = 45.55$, $p < 0.001$; and for the last question, $\chi^2(4) = 24.32$, $p < 0.001$. Participants selected Style as the easiest to read across all three questions, regardless of whether they were presented with the first, second, or third question. The Protein Lounge scored consistently highest for the second pathway, whereas CellDesigner had good performance for the third pathway.

4.1.2. Element Recognition. Participants were presented with pathways in which legends, titles, and explanations were deliberately excluded. Figure 14 summarizes the three questions presented to participants with respect to arrow type for Style. The order of presentation was randomized using Latin squares. Participants could select one of the following six answers to infer the meaning of the arrow: (1) positive regulation, (2) inhibition, (3) catalysis, (4) association/binding, (5) dissociation, and (6) translocation. Results indicate that 85.50% correctly recognized association/binding, 81.57% correctly identified translocation, and 92.11% correctly identified negative regulations.

Figure 15 summarizes the three questions presented to participants with respect to single elements. The order of presentation was randomized using Latin squares.

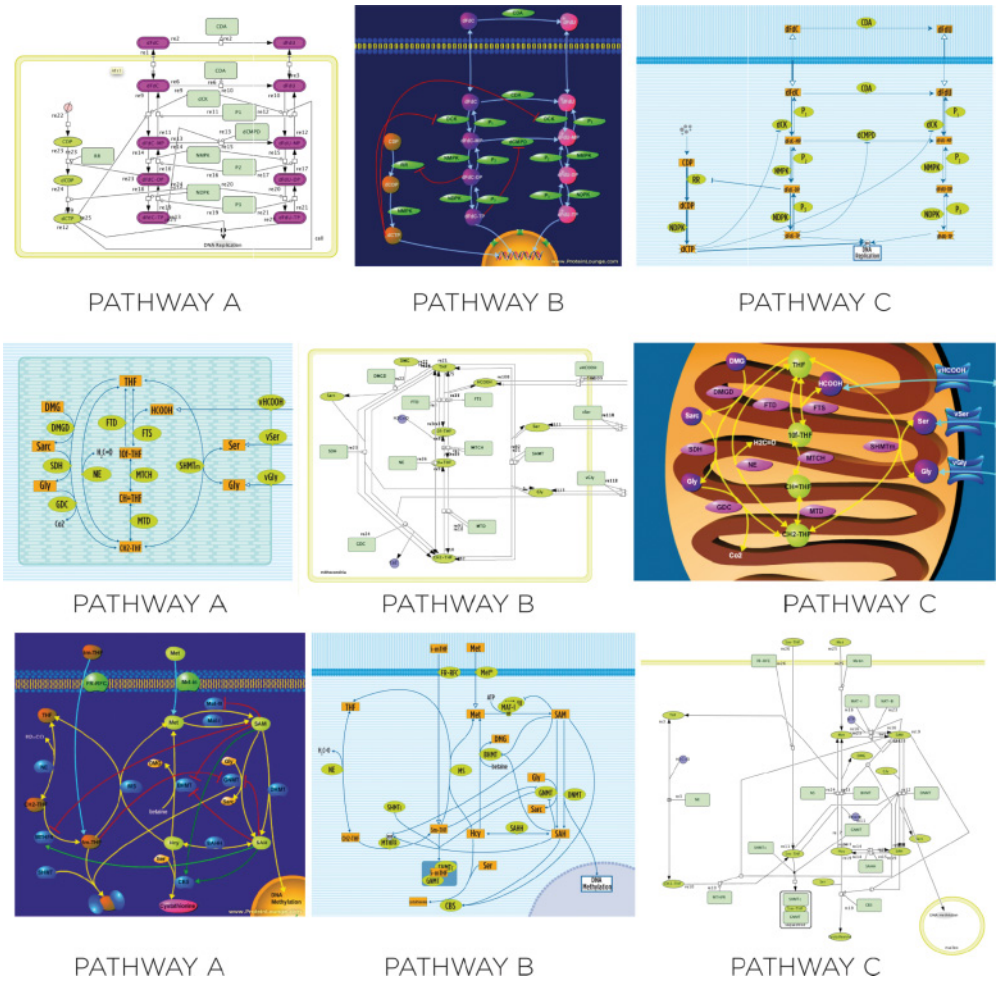


Fig. 12. Ranking questions: Gemcitabine metabolism illustrated in CellDesigner, Protein Lounge, and Style (first row, from left to right); folate cycle in mitochondria, illustrated in Style, CellDesigner, Protein Lounge (second row, from left to right); folate cycle in cytosol, illustrated in Protein Lounge, Style, and CellDesigner (third row, from left to right).

Participants could select one of the following five answers to infer the meaning of the elements: (1) enzyme, (2) metabolite, (3) complex, (4) link to another pathway, and (5) unknown precursor. Results indicate that 82.89% correctly identified metabolites, 80.26% correctly recognized enzymes, and 89.47% were able to correctly identify the undefined precursors.

Finally, we asked participants about their own interpretation using a smooth round enzyme that connects via an arrow to a spiky form (see Figure 16). Participants were asked to define the meaning of the spiky element in an open question. We interpreted answers as correct when they included one of the following phrases: (1) activated, (2) gets active, (3) becomes phosphorylated, or (4) changes to phosphorylated. Results indicate that 76.31% correctly identified the meaning of the illustration.

4.1.3. Computer Literacy and Statistical Programming Skills. The online survey collected data from the browser to complete the questionnaire. Data indicate that more than half the

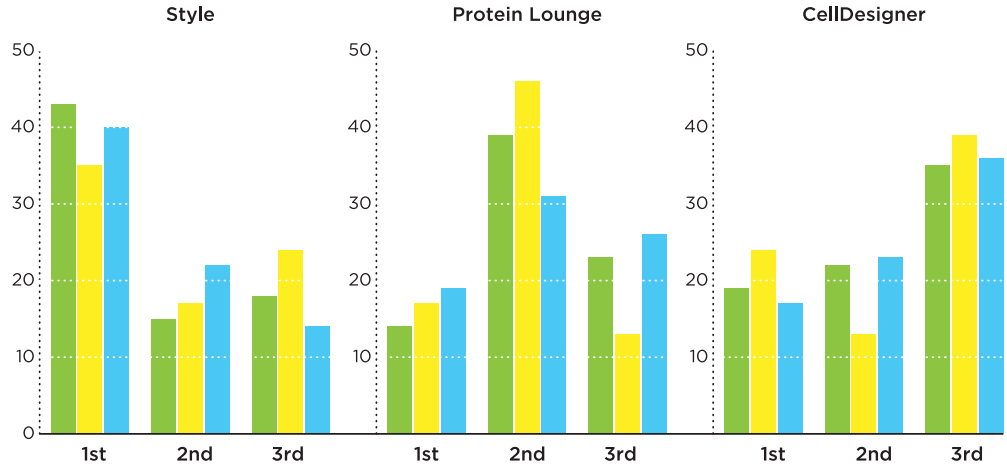


Fig. 13. A summary of the ranking scores grouped for each tool respectively (Style, Protein Lounge, and CellDesigner). The x-axis displays the three ranks (first, second, and third); the y-axis shows the number of ratings. The bar colors represent three pathways: green is gemcitabine, yellow is folate cycle in mitochondria, and blue is the folate cycle in cytosol.

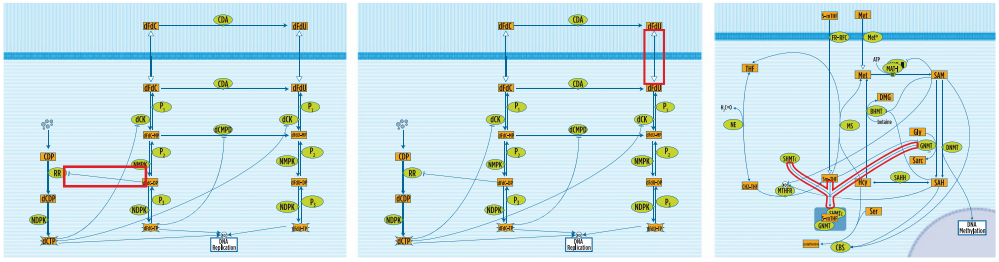


Fig. 14. Recognition task: inhibition (left), translocation (middle), association/binding (right).

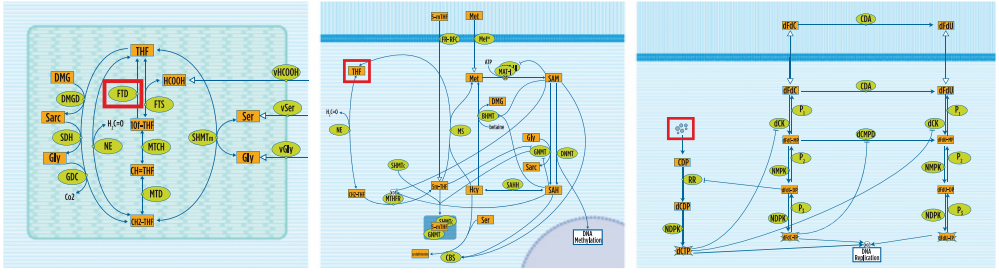


Fig. 15. Recognition task: membrane (left), small metabolite (middle), undefined precursor (right).

sample operated on a Windows system, more than 30% used Mac OS X, and 10% used Linux. When asked what operating system they were familiar with, 85.53% percent said Windows, 46.10% answered Mac OS X, and 31.58% said Linux (multiple selections were possible). Indeed, more than a third, (36.84%) were familiar with two operating systems, more than 10% were familiar with all three operating systems, and half of the sample were familiar with only a single operating system.

Participants were asked whether they were experts with the R statistics package (which a poll at bioinformatics.org with more than 1,700 respondents identified as the

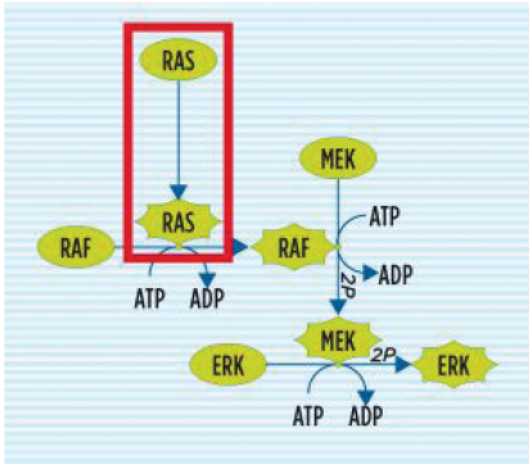


Fig. 16. Recognition task: participants had to infer the activation of an element without being given a legend or descriptive aid on the task.

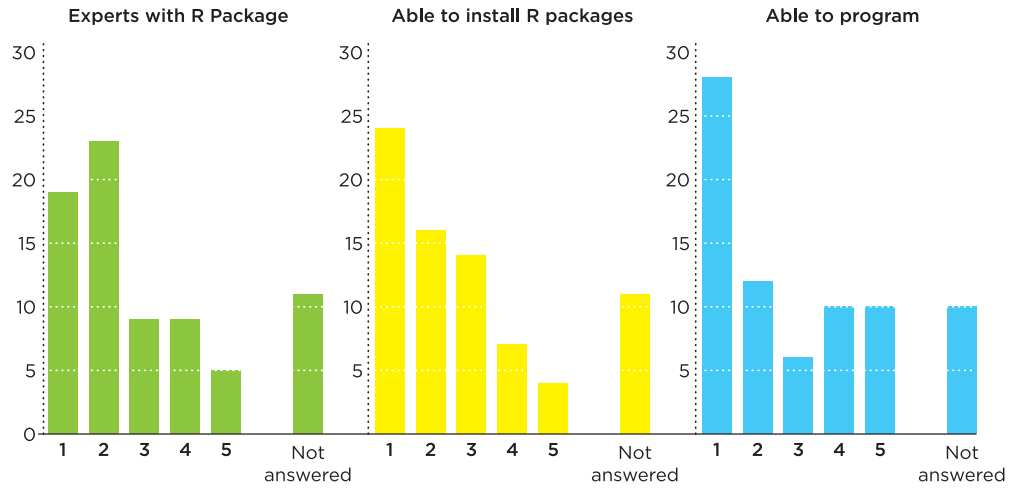


Fig. 17. Self-assessed programming skills of participants answered on a 5-bar Likert-scale from 1 (strongly disagree) to 5 (strongly agree) passing through 2 (disagree), 3 (neutral), and 4 (agree).

most used language for math/statistics), a free software package popular in the life science community used to analyze data. To answer the questions, subjects provided responses ranging from strongly agree to strongly disagree on a 5-bar Likert-scale.

We counted the answers Strongly Agree (5), Agree (4), and Neutral (3) as indicating researchers having expertise or experience in R and feeling comfortable with the software (see Figure 17), resulting in 30.26% of our sample. This percentage would exclude any individuals who indicated "Disagree" with this question to indicate only a basic knowledge of R and thus should only capture comfortable R users rather than basic R users. Similarly, we counted the answers for researchers feeling comfortable installing additional packages for the software R, resulting in 32.89%. Finally, we asked subjects to assess their programming skills. As the histogram in Figure 17 indicates, answers were of the same nature: counting from Strongly Agree to Neutral, 32.89% of

participants assess themselves as having some programming skills. Overall, about 30% of the surveyed participants had programming skills.

4.2. Discussion

Our results agree with insights from the requirement analysis by Saraiya et al. in the sense that Protein Lounge (representing a pathway drawing tool that integrates biological contexts similar to BioCarta) is rated consistently higher than cartoons made in CellDesigner using current SBGN notation [Saraiya et al. 2005]. Our data suggest that SBGN is hard to read because it was consistently rated as the least readable pathway in the presented sample. The results for Style being rated higher than Protein Lounge validates our guidelines for implementing interfaces. Indeed, our data suggest that Style is a diagrammatic modeling language with a concise semantic that is competitive with textbook illustrations.

For the recognition tasks, participants had to choose one out of six alternatives. The survey incorporated also one recognition task with an open text answer. Data suggest that the recognition with open text answer is lower than the recognition with predefined alternatives; however, we used a very conservative counting method, integrating correct answers only if one of the four options was explicitly written out. Overall, the data suggest that Style uses symbols and signs that are easy to read and self-explanatory, thus reinforcing the validation of our design.

Finally, the fact that the researchers from our center have started using Style as the de facto standard to illustrate biological processes for their papers, presentations, and posters is a promising first step.

5. CONCLUSION

This article presented a survey of existing open source graphical modeling software tools for reaction pathways with 2D (possibly nested) compartments and that are equipped with a simulation engine. We classified these tools according to the skills required to access them, such as programming, specific domain language familiarity, or simple computer literacy. We then discussed usability characteristics, collecting the most desirable features into a specification for a graphical language intended as a guideline for future implementations. We performed an online survey evaluating the proposed specification.

The current standard modeling languages seem to have low usability due to the large number of graphical symbols used to produce unambiguous diagrams. In our proposal (Style), we focused on two aspects: a minimal alphabet of graphical symbols to reduce clutter and symbols and shapes chosen to resemble as much as possible illustrations from biological textbooks. Figure 16 illustrates the MAPK pathway in Style that, similar to the textbook example, highlights the proteins using green shapes, making them more visually impactful. Furthermore, the text-only description of the reaction from ATP to ADP makes them less visually important and less cluttered. Finally, we propose the application of arrow labeling (here with 2P) to signal that the same reaction will be repeated twice before the element goes into an active state. Behind the scenes, there is the intermediate state of Mek being once phophorylated, which a user might decide to hide from the model for more clarity. In this way, Style reaches a high level of compactness similar to textbook illustrations while maintaining a precise meaning and permitting the mapping of such a model to a formalism that is suitable for simulation.

In addition, our online survey showed that Style is competitive to textbook illustrations and was constantly rated higher than SBGN. Our data also suggest that Style is intuitive to read and self-explanatory for biologists.

A current main limitation of bioinformatics techniques is the need for specialists to apply them. An intuitive graphical interface for modeling pathways, one that is formally designed to be unambiguous and automatically translated in the executable representations of systems, could improve the dissemination of modeling and simulation techniques for reaction pathways with 2D compartments. Our design study is currently under implementation in a prototype that will be made available to the scientific community. To broaden the usage of simulation techniques in biology, we are also designing a second layer of information that can be accessed from the modeling interface to feed models with quantitative parameters. For this second layer, we adopt a table-based view that we have already implemented successfully in our COSBILAB prototype (available from www.cosbi.eu). The connection of unambiguous graphical representations with quantitative information provides everything that is needed to automatically generate model representations ready for simulation using a range of different tools (varying from reaction-based to language/rule-based models).

ACKNOWLEDGMENTS

We thank the whole COSBI team for fruitful discussions and extensive testing of the design of Style.

REFERENCES

Ozgur E, Akman, David A. Rand, Paul E. Brown, and Andrew J. Millar. 2010. Robustness from flexibility in the fungal circadian clock. *BMC Systems Biology* 4, 1 (2010), 88.

B. Alberts. 2008. *Molecular Biology of the Cell: Reference Edition*. Number v. 1 in Molecular Biology of the Cell: Reference Edition. Taylor & Francis.

Boanerges Aleman-Meza, Yihai Yu, Heinz-Bernd Schüttler, Jonathan Arnold, and Thiab R. Taha. 2009. KINSOLVER: A simulator for computing large ensembles of biochemical and gene regulatory networks. *Computers & Mathematics with Applications* 57, 3 (2009), 420–435.

Steven S. Andrews, Nathan J. Addy, Roger Brent, and Adam P. Arkin. 2010. Detailed simulations of cell biology with Smoldyn 2.1. *PLoS Computational Biology* 6, 3 (2010), e1000705.

John A. Bachman and Peter Sorger. 2011. New approaches to modeling complex biochemistry. *Nature Methods* 8, 2 (2011), 130.

Frank T. Bergmann and Herbert M. Sauro. 2006. SBW-a modular framework for systems biology. In *Proceedings of the 38th Conference on Winter Simulation*. Winter Simulation Conference, 1637–1645.

William J. Bosl. 2007. Systems biology by the rules: Hybrid intelligent systems for pathway modeling and discovery. *BMC Systems Biology* 1, 1 (2007), 13.

Dennis Bray, Robert B. Bourret, and Melvin I. Simon. 1993. Computer simulation of the phosphorylation cascade controlling bacterial chemotaxis. *Molecular Biology of the Cell* 4, 5 (1993), 469.

Henri Casanova, Thomas M. Bartol, Joel Stiles, and Francine Berman. 2001. Distributing MCell simulations on the Grid. *International Journal of High Performance Computing Applications* 15, 3 (2001), 243–257.

Deepak Chandran, Frank T. Bergmann, Herbert M. Sauro, and others. 2009. TinkerCell: Modular CAD tool for synthetic biology. *Journal of Biological Engineering* 3, 1 (2009), 19.

R. Cheong, A. Hoffmann, and A. Levchenko. 2008. Understanding NF-κ B signaling via mathematical modeling. *Molecular Systems Biology* 4, 192 (2008).

Federica Ciocchetta, Adam Duguid, Stephen Gilmore, Maria Luisa Guerriero, and Jane Hillston. 2009. The Bio-PEPA tool suite. In *Proceedings of the 6th International Conference on the Quantitative Evaluation of Systems (QEST '09)*. IEEE, 309–310.

Robert Clewley. 2012. Hybrid models and biological model reduction with PyDSTool. *PLoS Computational Biology* 8, 8 (2012), e1002628.

Andy Cockburn, Amy Karlson, and Benjamin B. Bederson. 2008. A review of overview + detail, zooming, and focus + context interfaces. *ACM Computing Surveys (CSUR)* 41, 1 (2008), 2.

J. Colvin, M. I. Monine, J. R. Faeder, W. S. Hlavacek, D. D. Von Hoff, and R. G. Posner. 2009. Simulation of large-scale rule-based models. *Bioinformatics* 25, 7 (Apr 2009), 910–917.

L. Dematté, C. Priami, and A. Romanel. 2008. The Beta Workbench: A computational tool to study the dynamics of biological systems. *Briefings in Bioinformatics* 9, 5 (2008), 437–449.

Pawan Dhar, Tan Chee Meng, Sandeep Somani, Li Ye, Anand Sairam, Mandar Chitre, Zhu Hao, and Kishore Sakharkar. 2004. Cellware: A multi-algorithmic software for computational systems biology. *Bioinformatics* 20, 8 (2004), 1319–1321.

Niraj Dudani, Subhasis Ray, Siji George, and Upinder S. Bhalla. 2009. Multiscale modeling and interoperability in MOOSE. *Neuroscience* 10, Suppl 1 (2009), 54.

Peter Eades, Wei Lai, Kazuo Misue, and Kozo Sugiyama. 1991. *Preserving the Mental Map of a Diagram*. International Institute for Advanced Study of Social Information Science, Fujitsu Limited.

Florian Erhard, Caroline C. Friedel, and Ralf Zimmer. 2008. FERN–A Java framework for stochastic simulation and evaluation of reaction networks. *BMC Bioinformatics* 9, 1 (2008), 356.

Rudolf Fleischer and Colin Hirsch. 2001. Graph drawing and its applications. In *Drawing Graphs*. M. Kaufmann and D. Wagner, Eds., Springer, 1–22.

Yaniv Frishman and Ayellet Tal. 2004. Dynamic drawing of clustered graphs. In *Information Visualization, 2004. INFOVIS 2004. IEEE Symposium on*. IEEE, 191–198.

Akira Funahashi, Yukiko Matsuoka, Akiya Jouraku, Mineo Morohashi, Norihiro Kikuchi, and Hiroaki Kitano. 2008. CellDesigner 3.5: A versatile modeling tool for biochemical networks. *Proceedings of the IEEE* 96, 8 (2008), 1254–1265.

T. D. Garvey, P. Lincoln, C. J. Pedersen, D. Martin, and M. Johnson. 2003. BioSPICE: Access to the most current computational tools for biologists. *OMICS* 7, 4 (2003), 411–420.

Nail M. Gizzatkulov, Igor I. Goryanin, Eugeny A. Metelkin, Ekaterina A. Mogilevskaya, Kirill V. Peskov, and Oleg V. Demin. 2010. DBSolve Optimum: A software package for kinetic modeling which allows dynamic visualization of simulation results. *BMC Systems Biology* 4, 1 (2010), 109.

Gerd Grünert and Peter Dittrich. 2011. Using the SRSim software for spatial and rule-based modeling of combinatorially complex biochemical reaction systems. In *Membrane Computing*. M. Gheorghe, T. Hinze, G. Paun, G. Rozenberg, and A. Salomaa, Eds., Springer, 240–256.

Anthony D. Hill, Jonathan R. Tomshine, Emma M. B. Weeding, Vassilios Sotiropoulos, and Yiannis N. Kaznessis. 2008. SynBioSS: The synthetic biology modeling suite. *Bioinformatics* 24, 21 (2008), 2551–2553.

S. Hoops, S. Sahle, R. Gauges, C. Lee, J. Pahle, N. Simus, M. Singhal, L. Xu, P. Mendes, and U. Kummer. 2006. COPASI–A COmplex PAthway SImulator. *Bioinformatics* 22, 24 (Dec 2006), 3067–3074.

Xiaodi Huang, Wei Lai, A. S. M. Sajeev, and Junbin Gao. 2007. A new algorithm for removing node overlapping in graph visualization. *Information Sciences* 177, 14 (2007), 2821–2844.

Michael Hucka, Andrew Finney, Herbert M. Sauro, Hamid Bolouri, John C. Doyle, Hiroaki Kitano, Adam P. Arkin, Benjamin J. Bornstein, Dennis Bray, Athel Cornish-Bowden, and others. 2003. The systems biology markup language (SBML): A medium for representation and exchange of biochemical network models. *Bioinformatics* 19, 4 (2003), 524–531.

Jonathan R. Karr, Jayodita C. Sanghvi, Derek N. Macklin, Miriam V. Gutschow, Jared M. Jacobs, Benjamin Bolival Jr., Nacyra Assad-Garcia, John I. Glass, and Markus W. Covert. 2012. A whole-cell computational model predicts phenotype from genotype. *Cell* 150, 2 (2012), 389–401.

Boris N. Kholodenko. 2006. Cell-signalling dynamics in time and space. *Nature Reviews Molecular Cell Biology* 7 (2006), 165–176.

Andrzej M. Kierzek. 2002. STOCKS: STOChastic kinetic simulations of biochemical systems with Gillespie algorithm. *Bioinformatics* 18, 3 (2002), 470–481.

Hiroaki Kitano. 2002. Systems biology: A brief overview. *Science* 295, 5560 (2002), 1662–1664.

F. Kolpakov, M. Puzanov, and A. Koshukov. 2006. BioUML: Visual modeling, automated code generation and simulation of biological systems. In *Proceedings of the 5th International Conference on Bioinformatics of Genome Regulation and Structure*. 281–285.

Markus Koschorreck and Ernst D. Gilles. 2008. ALC: Automated reduction of rule-based models. *BMC Systems Biology* 2, 1 (2008), 91.

J. Krumsiek, S. Polsterl, D. M. Wittmann, and F. J. Theis. 2010. Odefy–from discrete to continuous models. *BMC Bioinformatics* 11 (2010), 233.

Hiroyuki Kurata, Kentaro Inoue, Kazuhiro Maeda, Koichi Masaki, Yuki Shimokawa, and Quanyu Zhao. 2007. Extended CADLIVE: A novel graphical notation for design of biochemical network maps and computational pathway analysis. *Nucleic Acids Research* 35, 20 (2007), e134–e134.

N. Le Novère, M. Hucka, H. Mi, S. Moodie, F. Schreiber, A. Sorokin, E. Demir, K. Wegner, M. I. Aladjem, S. M. Wimalaratne, F. T. Bergman, R. Gauges, P. Ghazal, H. Kawaji, L. Li, Y. Matsuoka, A. Villéger, S. E. Boyd, L. Calzone, M. Courtot, U. Dogrusoz, T. C. Freeman, A. Funahashi, S. Ghosh, A. Jouraku, S. Kim, F. Kolpakov, A. Luna, S. Sahle, E. Schmidt, S. Watterson, G. Wu, I. Goryanin, D. B. Kell, C. Sander,

H. Sauro, J. L. Snoep, K. Kohn, and H. Kitano. 2009. The systems biology graphical notation. *Nature Biotechnology* 27 (2009), 735–741.

Nicolas Le Novere and Thomas Simon Shimizu. 2001. STOCHSIM: Modelling of stochastic biomolecular processes. *Bioinformatics* 17, 6 (2001), 575–576.

Dong-Yup Lee, Choamun Yun, Ayoun Cho, Bo Kyeng Hou, Sunwon Park, and Sang Yup Lee. 2006. WebCell: A web-based environment for kinetic modeling and dynamic simulation of cellular networks. *Bioinformatics* 22, 9 (2006), 1150–1151.

W. J. Longabaugh. 2012. BioTapestry: A tool to visualize the dynamic properties of gene regulatory networks. *Methods in Molecular Biology (Clifton, NJ)* 786 (2012), 359.

Carlos F. Lopez, Jeremy L. Muhlich, John A. Bachman, and Peter K. Sorger. 2013. Programming biological models in Python using PySB. *Molecular Systems Biology* 9, 1 (2013).

Aneil Mallavarapu, Matthew Thomson, Benjamin Ullian, and Jeremy Gunawardena. 2007. Modular model building. *arXiv preprint arXiv:0710.3421* (2007).

Anthea Maton, David Lahart, Jean Hopkins, Maryanna Quon Warner, Susan Johnson, and Jill D. Wright. 1997. *Cells: Building Blocks of Life*. Pearson Prentice Hall.

Martin Meier-Schellersheim, Xuehua Xu, Bastian Angermann, Eric J. Kunkel, Tian Jin, and Ronald N. Germain. 2006. Key role of local regulation in chemosensing revealed by a new molecular interaction-based modeling method. *PLoS Computational Biology* 2, 7 (2006), e82.

Pedro Mendes. 1993. GEPASI: A software package for modelling the dynamics, steady states and control of biochemical and other systems. *Computer Applications in the Biosciences: CABIOS* 9, 5 (1993), 563–571.

John A. Miller, Andrew F. Seila, and Xuewei Xiang. 2000. The JSIM web-based simulation environment. *Future Generation Computer Systems* 17, 2 (2000), 119–133.

Bud Mishra, Marco Antoniotti, Salvatore Paxia, and Nadia Ugel. 2005. Simpathica: A computational systems biology tool within the valis bioinformatics environment. *Computational Systems Biology* (2005).

I. I. Moraru, J. C. Schaff, B. M. Slepchenko, M. L. Blinov, F. Morgan, A. Lakshminarayana, F. Gao, Y. Li, and L. M. Loew. 2008. Virtual cell modelling and simulation software environment. *IET Systems Biology* 2, 5 (2008), 352–362.

M. J. Morine, A. C. Tierney, B. van Ommen, H. Daniel, S. Toomey, I. M. F. Gjelstad, I. C. Gormley, P. Prez-Martinez, C. A. Drevon, J. L-pez-Miranda, and H. M. Roche. 2011. Transcriptomic coordination in the human metabolic network reveals links between n-3 fat intake, adipose tissue gene expression and metabolic health. *PLoS Computational Biology* 7, 11 (2011).

Christoph Müssel, Martin Hopfensitz, and Hans A. Kestler. 2010. BoolNet: An R package for generation, reconstruction and analysis of Boolean networks. *Bioinformatics* 26, 10 (2010), 1378–1380.

C. J. Myers, N. Barker, K. Jones, H. Kuwahara, C. Madsen, and N. P. Nguyen. 2009. iBioSim: A tool for the analysis and design of genetic circuits. *Bioinformatics* 25, 21 (Nov 2009), 2848–2849.

Christopher R. Myers, Ryan N. Gutenkunst, and James P. Sethna. 2007. Python unleashed on systems biology. *Computing in Science & Engineering* 9, 3 (2007), 34–37.

Gabriele Neyer. 2001. Map labeling with application to graph drawing. In *Drawing Graphs*. M. Kaufmann and D. Wagner, Eds., Springer, 247–273.

Brett G. Olivier and Jacky L. Snoep. 2004. Web-based kinetic modelling using JWS Online. *Bioinformatics* 20, 13 (2004), 2143–2144.

Julien F. Ollivier, Vahid Shahrezaei, and Peter S. Swain. 2010. Scalable rule-based modelling of allosteric proteins and biochemical networks. *PLoS Computational Biology* 6, 11 (2010), e1000975.

Andrew Phillips. 2009. A visual process calculus for biology. *Symbolic Systems Biology: Theory and Methods.* Jones and Bartlett.

Corrado Priami. 2009. Algorithmic systems biology. *CACM* 52, 5 (2009), 80–88.

C. Priami, A. Regev, E. Shapiro, and W. Silverman. 2001. Application of a stochastic name-passing calculus to representation and simulation of molecular processes. *Information Processing Letter* 80 (2001), 25–31.

Helen C. Purchase, Jo-Anne Allder, and David Carrington. 2001. User preference of graph layout aesthetics: A UML study. In *Graph Drawing*. M. Kaufmann and D. Wagner, Eds., Springer, 5–18.

S. Ramsey, D. Orrell, and H. Bolouri. 2005. Dizzy: Stochastic simulation of large-scale genetic regulatory networks. *Journal of Bioinformatics Computing Biology* 3, 2 (Apr 2005), 415–436.

P. Rangamani, A. Lipshtat, E. U. Azeloglu, R. C. Calizo, M. Hu, S. Ghassemi, J. Hone, S. Scarlata, S. R. Neves, and R. Iyengar. 2013. Decoding information in cell shape. *Cell* 154, 6 (2013), 1356–69.

Andre S. Ribeiro, Daniel A. Charlebois, and Jason Lloyd-Price. 2007. CellLine, a stochastic cell lineage simulator. *Bioinformatics* 23, 24 (2007), 3409–3411.

Andre S. Ribeiro and Jason Lloyd-Price. 2007. SGN Sim, a stochastic genetic networks simulator. *Bioinformatics* 23, 6 (2007), 777–779.

S. K. Sadiq, R. Guixa-Gonzalez, E. Dainese, M. Pastor, G. De Fabritiis, and J. Selent. 2013. Molecular modeling and simulation of membrane lipid-mediated effects on GPCRs. *Current Medicinal Chemistry* 20, 1 (2013), 22–38.

Purvi Saraiya, Chris North, and Karen Duca. 2005. Visualizing biological pathways: Requirements analysis, systems evaluation and research agenda. *Information Visualization* 4, 3 (June 2005), 191–205. DOI: http://dx.doi.org/10.1057/palgrave.ivs.9500102

A. M. Smith, W. Xu, Y. Sun, J. R. Faeder, and G. E. Marai. 2012. RuleBender: Integrated modeling, simulation and visualization for rule-based intracellular biochemistry. *BMC Bioinformatics* 13 Suppl 8 (2012), S3.

Marc T. Vass, Clifford A. Shaffer, Naren Ramakrishnan, Layne T. Watson, and John J. Tyson. 2006. The JigCell model builder: A spreadsheet interface for creating biochemical reaction network models. *IEEE/ACM Transactions on Computational Biology and Bioinformatics on* 3, 2 (2006), 155–164.

Michael Weber and Ekkart Kindler. 2003. The petri net kernel. *Petri Net Technology for Communication-Based Systems* (2003), 109–123.

Katja Wegner, Johannes Knabe, Mark Robinson, Attila Egri-Nagy, Maria Schilstra, and Chrystopher Nehaniv. 2007. The NetBuilder'project: Development of a tool for constructing, simulating, evolving, and analysing complex regulatory networks. *BMC Systems Biology* 1, Suppl 1 (2007), P72.

Hans V. Westerhoff and Bernhard O. Palsson. 2004. The evolution of molecular biology into systems biology. *Nature Biotechnology* 22, 10 (2004), 1249–1252.

J. Zheng, D. Zhang, P. F. Przytycki, R. Zielinski, J. Capala, and T. M. Przytycka. 2010. SimBoolNet—a Cytoscape plugin for dynamic simulation of signaling networks. *Bioinformatics* 26, 1 (2010), 141–142.

Received July 2013; revised March 2014; accepted June 2014